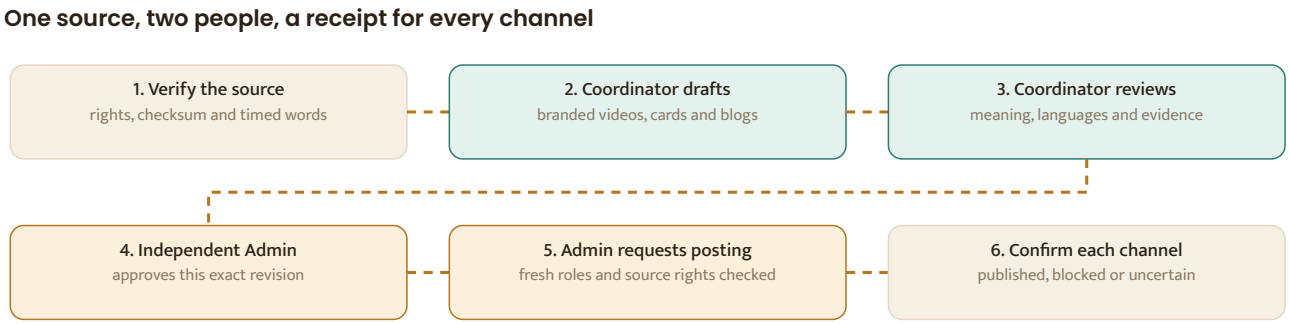
Intelligent Content Studio operations

Studio adds a durable generation, review and publication workflow to Distribution Hub. The Content Coordinator prepares the package. An independent Admin reviews it and requests publication. Generation creates drafts; it never approves its own work or silently posts a package.

Studio's finalized combination reuses existing keys: **Sarvam Saaras v4** transcription and **Gemini 3.5 Flash** drafting. The local preview remains TEST; drafts need qualified native-language review and independent Admin approval. See publishing providers for connector requirements.

Operator workflow



Walk the approval boundary. Coordinator changes require another Admin review; TEST publication ends as simulated and never posts to social accounts.

1. Open an uploaded source in Distribution Hub and use its Studio panel. Select output languages, provide optional audience/editorial guidance and select captions. The UI considers all supported channels and shows why a channel was skipped.
2. The Content Coordinator starts generation. The worker reads the exact stored source checksum, obtains a timed transcript, drafts source-linked content and renders branded artifacts. Short/reel selections retain complete contiguous source segments, with a maximum three-minute selected range.
3. Review the videos, cards, full blog text, source evidence and language warnings. Include or exclude individual outputs. Post/blog copy edits invalidate its generated asset and require regeneration before submission. Video changes are made through regeneration guidance. A changed package requires review again.
4. Submit for Admin review. The Admin may approve, reject with feedback or discard. Approval is tied to the current package and source revisions. A contributor cannot approve their own package; source and editorial independence checks also remain enforced.
5. The Admin requests publication of the approved included outputs. Each channel receives its own durable publication record. The panel distinguishes queued, processing, published, simulated, blocked, failed and uncertain states. Only an actual provider receipt or the completed internal website publication becomes published.

Source changes, withdrawn rights, stale approval, revoked roles and wrong-world access block the affected work. Generation, feedback, reviews and publication attempts remain auditable. A new generation request does not reuse an old approval.

Generated formats, languages and branding

YouTube receives Shorts; Instagram, Facebook and TikTok receive Reels; X, Threads, LinkedIn, Pinterest, Telegram and Bluesky receive post/card drafts; Reddit and the Ashrama website receive long-form text. Suitability is part of drafting: for example, local drafting skips LinkedIn for devotional recordings outside the service/community category.

Source/output language codes are `en`, `hi`, `te`, `kn`, `ta`, `ml`, `gu` and `sa`. Local drafting produces source-language excerpts and format adaptations. It does not claim AI interpretation or translation. Gemini and OpenAI planning support requested translated copy and source-segment subtitle translations when explicitly configured. In the Sarvam/Gemini configuration, Sarvam recognizes the original-language speech and Gemini drafts the channel copy and requested translations. This workflow does not call Sarvam Translate. Provider quality and source-language recognition still require review; supporting a language code is not evidence that every recording in that language will be accurately recognized. A qualified native reviewer must check names, Sanskrit verses, language switches and the meaning of every translated excerpt before approval.

The selected source audio remains unchanged. With captions enabled, the selected output-language captions are burned into the generated video and also retained as WebVTT. Translated videos additionally retain a source-language WebVTT track. Translation keeps source segment IDs and timing. Long cues are paged for display rather than dropping words. The preview can expose the available caption tracks; external video publishing does not depend on a provider accepting sidecar files.

Media uses the configured Ashrama logo, fixed Ashrama name and SGS Datta Sannidhi signature. The app appends the canonical signature to AI copy if missing, never truncates it, and validates the resulting channel limits. Blogs include attribution and master lineage; outputs retain source IDs, language, channel, checksum and immutable asset identity. Human approval still requires checking complete meaning, attribution, translations, legibility and the source; structural validation cannot prove theological or semantic accuracy.

Source size and transcript preparation

The existing upload limit is **50 MiB per master**. Studio does not add streaming uploads or remove that limit. The supplied Episode 249 raw master is approximately 410 MiB, so it cannot be uploaded unchanged through this route.

For a local demonstration with a larger recording, retain the original master intact and produce a separately named, full-duration optimized proxy below the upload limit. Record the original checksum, proxy checksum, transformation settings and full-duration comparison in the evidence. Register the proxy as the actual uploaded source; its checksum is the Studio source checksum. Do not represent its bytes as identical to the original, silently overwrite the original file, or trim the teaching to meet the upload limit. Image/audio quality of the proxy still needs review.

Transcript selection order is: current approved timed transcript; explicitly selected Sarvam or OpenAI timed transcription; otherwise a source-checksum-bound private cache or configured local transcription executable. Choosing a remote transcription provider bypasses the cache so that an older local transcript cannot be presented as a new provider result. Without any applicable source, generation fails with a transcription configuration error. A cached or machine transcript is labelled machine provenance, not human approval. Sarvam uses the full recording through its asynchronous batch API and returns phrase/sentence timing; Studio does not invent word-level timestamps.

An optional private cache uses `<source-sha256>.json` beneath `STUDIO_TRANSCRIPT_CACHE_ROOT`:

```
{
 "sourceSha256": "EXACT_UPLOADED_SOURCE_SHA256",
 "raw": {
   "language": "en",
   "segments": [{ "start": 0, "end": 1.5, "text": "Timed source words." }]
 }
}
```

The file must be private and is bounded to 20 MiB. The outer checksum must match the uploaded source. Seconds in `raw.segments` are normalized to integer milliseconds; overlapping, reversed or out-of-duration cues are refused. A local transcription executable must accept `--`

`input`, `--language` and `--output` arguments and write the same timed transcript shape. It runs as an explicitly configured local program, without shell interpolation.

Worker and generation configuration

Apply the additive Hub migration `0011_luxuriant_sway.sql` through the existing dedicated migration-role workflow before starting this version. Reapply the Hub runtime grants through the existing procedure. Migrations do not run on worker boot. Core/Hub identity and role configuration must agree on the TEST or production world.

`.env.example` selects Sarvam `saaras:v4` and Gemini `gemini-3.5-flash`, with generation and publishing off. The table shows enabling settings, not defaults.

Setting	Purpose
`DISTRIBUTION_HUB_ENABLED=true`	Enables the Hub runtime gate
`DISTRIBUTION_HUB_STAGE=TEST`, `LIMITED` or `ON`	Selects TEST or production operation; TEST database access stays loopback
`DISTRIBUTION_DATABASE_URL`	Dedicated Hub database connection
`STUDIO_WORKER_ENABLED=true`	Starts the separate Studio worker
`STUDIO_GENERATION_ENABLED=true`	Allows generation readiness
`STUDIO_INTELLIGENCE_PROVIDER=local`, `gemini` or `openai`	Chooses source excerpts or the selected AI drafting provider
`STUDIO_OPENAI_API_KEY`, `STUDIO_OPENAI_MODEL`	Required private credentials and explicit model for OpenAI planning
`STUDIO_GEMINI_API_KEY`, `STUDIO_GEMINI_MODEL`	Private Gemini key and explicit saved model; supplied in memory by the reuse bridge below
`STUDIO_TRANSCRIPTION_PROVIDER=sarvam` or `openai`	Selects remote timed audio recognition and bypasses the local transcript cache
`STUDIO_SARVAM_API_KEY`, `STUDIO_SARVAM_MODEL`	Private Sarvam key and speech model; reuse bridge defaults to `saaras:v4`, with `saaras:v3` also supported
`STUDIO_SARVAM_KEYTERMS`	Optional JSON array of up to 50 glossary terms, each at most 64 characters; requires `saaras:v4`
`STUDIO_WHISPER_COMMAND`	Absolute path to the optional local timed-transcription executable

Setting	Purpose
`STUDIO_TRANSCRIPT_CACHE_ROOT`	Absolute path to an optional private checksum-bound transcript cache
`STUDIO_BRAND_LOGO_PATH`	Absolute path to the approved Ashrama logo
`DISTRIBUTION_FFMPEG_PATH`, `DISTRIBUTION_FFPROBE_PATH`	Local media executables
`STUDIO_SCRATCH_ROOT`	Private local generation scratch directory; falls back to the configured local asset root

External AI processing additionally requires current public, worldwide source rights covering the requested channels and valid release/expiry dates. This authorization is checked by the repository and again during the work. Private content is not sent to an AI provider merely because a key exists. The OpenAI Responses request uses `store:false`; that parameter alone is not a claim of zero provider retention, and it is not a setting for Sarvam or Gemini. Provider selection and handling of source material remain deliberate deployment decisions.

Reuse the configured Sarvam and Gemini services

`ops/studio-configured-providers.ts` is an operator-side startup bridge. It reads the app's existing `provider_credentials` records, selects active Sarvam and Gemini instances and opens their stored credentials with the app's existing sealing key. It performs no configuration writes. The isolated Distribution packages do not import the core credential store, and the browser never receives either API key. The bridge passes keys only to the worker's process environment; it creates no plaintext credential export and removes the core database URL and sealing key from the child environment.

Bridge setting	Purpose
`STUDIO_CORE_DATABASE_URL`	Core provider-store connection; falls back to the app's `DATABASE_URL`; TEST requires a loopback host
`APP_SECRETS_KEY`	The existing core app sealing key; available only to the operator bootstrap
`STUDIO_SARVAM_PROVIDER`	Exact saved Sarvam instance name, such as `sarvam-chat`
`STUDIO_GEMINI_PROVIDER`	Exact saved Gemini instance name, such as `gemini-chat`
`STUDIO_SARVAM_MODEL`	Optional explicit speech-model override; otherwise `config.asrModel`, a saved Saaras `config.model`, or `saaras:v4`
`STUDIO_GEMINI_MODEL`	Explicit override first; otherwise `config.studioModel`, then legacy `config.model`
`STUDIO_SARVAM_CONFIGURED`, `STUDIO_GEMINI_CONFIGURED`	Non-secret readiness flags projected only after the bridge successfully loads both configurations

Select an exact instance when multiple instances are active. A disabled or missing selection fails without changing accounts. The finalized Hub settings are Sarvam `config.asrModel=saaras:v4` and Gemini `config.studioModel=gemini-3.5-flash`. These scoped fields reuse existing credentials and leave each provider's shared chat `config.model` unchanged. Gemini resolution is explicitly `STUDIO_GEMINI_MODEL`, then `config.studioModel`, then legacy `config.model`. A Sarvam chat model such as `sarvam-105b` is not used for recognition. Choosing this combination does not certify accuracy for every recording; native review remains required.

Load the existing core app environment and the dedicated Hub worker settings through the process supervisor. Set the two instance selectors, then run from the repository root:

```
 node --import tsx ops/studio-configured-providers.ts --check
node --import tsx ops/studio-configured-providers.ts --readiness
node --import tsx ops/studio-configured-providers.ts
```

`--check` verifies that both selected configurations can be read and decrypted. It prints only provider modes, model names and readiness flags. `--readiness` prints the same projection for configuring web readiness without provider keys. Neither command calls a vendor or certifies credit, model access or source quality. The last command launches the Studio worker. Supervisors can import `loadStudioConfiguredProviderEnv`, `studioConfiguredProviderReadiness` and `studioConfiguredWorkerEnv` to apply these boundaries. Do not serialize the loader's full result.

Every 30 seconds, the bootstrap rereads provider settings. Disabling/removing a selected provider, changing its effective key/model/identity or failing a recheck stops the worker; it exits with code 78 after graceful shutdown. Restart to load updated settings and refresh web readiness. Custom supervisors must install `monitorStudioConfiguredProviders` and cancel it on shutdown. Rechecks are read-only, and errors never expose credentials.

Sarvam receives audio extracted from the full uploaded source. Gemini receives that recording's audio and timed transcript for meaning and language-switching context; no video frames are supplied or visually analyzed. Original speech is preserved. Shorts/reels require complete contiguous source segments with unchanged timing. Posts/blogs may cite unique known segments in source order from separate supporting passages, preserving each passage's context. Uncertain or conflicting passages require review, never invented corrections. Captions and translations retain the human review gates.

Configure fresh role authority on the core app and Studio process:

Setting	Purpose
`STUDIO_WORKER_AUTHORIZATION_ENABLED=true`	Enables the signed Studio authority contract
`STUDIO_WORKER_AUTHORIZATION_KEY`	Shared private 64-character hexadecimal key
`STUDIO_CORE_ORIGIN`	Exact core origin; HTTP loopback in TEST, HTTPS outside loopback for production

`DISTRIBUTION_CORE_ORIGIN` is a supported origin fallback. The older worker authorization key is a TEST-only fallback. The versioned core endpoint is `/api/v1/studio-worker-authority`; identities from sign-in providers do not grant Studio permissions.

Supply these values through private process configuration, then start the worker:

```
pnpm --filter @satsang/distribution-worker studio:start
```

Generation and publication poll independently. The generation lease renews while source/role authority remains valid. Requests and media processing have bounded execution times; stopping or losing authorization cancels work. Gemini permits one bounded repair when complete JSON fails plan validation; transport, authentication and incomplete-response failures are not retried. Provider errors return stable codes without response bodies or credentials. A failed repair cannot create a successful package.

Storage, public media and publication readiness

Studio uses the same private asset-store interface as source uploads and Variant Studio. Local disk remains the default. Explicit S3 configuration uses the existing world-specific prefix/owner/checksum controls; see private storage operations. S3 is not required for local Studio generation and no AWS deployment is implied by configuring local storage.

Production publication additionally needs `STUDIO_PUBLISHING_ENABLED=true`. External connectors need the private `STUDIO_CHANNEL_CONFIG_FILE` described in publishing providers. Media publishing requires an exact HTTPS `STUDIO_PUBLIC_ORIGIN`, a private `STUDIO_MEDIA_SIGNING_SECRET` of at least 32 characters and the same origin in the provider configuration's trusted media origins.

The worker signs a restricted delivery URL for the specific publication/package revision/output/checksum. Providers receive that URL, not private disk paths or S3 bucket URLs. Its validity ends 24 hours after the publication record was created. The public route rechecks publication/source approval and serves only matching verified bytes. Expired media access blocks publication rather than sending an unusable link. An internal blog's public URL is `/distribution/blog/<publication-id>` and becomes readable only after its production publication completes.

Readiness has two distinct meanings. Generation readiness reports enabled provider configuration; it does not pre-run FFmpeg, inspect every source's transcript or certify its rights. Channel readiness reports configured account eligibility and delivery settings; it does not prove that a token has not expired. Actual jobs perform their own checks and expose a specific blocked or failed result. An absent connector file leaves external channels disconnected.

TEST jobs verify the same selected output/caption bytes and then record **SIMULATED**, with no external social posting or public blog publication. TEST does not mean external AI is automatically disabled: choosing Sarvam, Gemini or OpenAI can still make the explicitly configured, source-rights-gated recognition/drafting calls and consume provider credits. Use local drafting and approved/cache/local transcripts when no external processing is intended.

Failure and reconciliation

Generation failures preserve prior revision history and explicit feedback. A retry or regeneration produces another reviewable revision. The durable generation attempt cap prevents an endless crash/retry loop.

`STUDIO_EDITED_OUTPUT_OMITTED` means a newly generated plan skipped a still-requested channel/language containing a pending coordinator correction. Generation stops and preserves the prior revision instead of discarding that correction. Review the saved draft and revise generation guidance to retain the corrected output; removing a requested channel/language must be an explicit editorial decision. Corrections that are retained keep their original source-segment evidence.

Publication creation is never retried merely because the HTTP response or database commit acknowledgement was lost. The result becomes uncertain or its expired lease is recovered as uncertain. Known provider checkpoints permit status reads without recreating a post. Transient transport/429/5xx receipt failures are retried within the documented twenty-poll bound; authentication errors, invalid receipts and ambiguous creation require investigation.

To pause new publication, disable `STUDIO_PUBLISHING_ENABLED`. To stop the process, stop the Studio worker through the existing process supervisor. Preserve immutable source assets, package revisions, feedback, receipts and attempt history. Do not erase a checkpoint, reset an uncertain row to queued or bulk-delete source/derived storage to retry a publication.

Local evidence

Tests verify workflow safeguards. Preserve real-run provider/model provenance and source hashes; mocked requests do not establish native-language accuracy.